2025115062

Homework-7

1. Write a C program to construct an expression tree for a given expression.

Example: (a-b)+(c*d)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
struct node
{
   char data;     struct node
*left, *right;
};
struct node *stack[100]; int top
= -1; struct node
*createNode(char c)
{
   struct node *newnode =(struct node *)malloc(sizeof(struct node));
newnode->data = c;     newnode->left = NULL;     newnode->right =
NULL;     return newnode;

}
```

```
void push(struct node *n)
{
   stack[++top] = n;
}
struct node *pop()
{
   return stack[top--];
}
int main()
{
   char postfix[100];
   int i;
   struct node *root, *left, *right;
printf("Enter postfix expression: ");
scanf("%99s", postfix);     for (i = 0;
postfix[i] != '\0'; i++)
   {
      if (isalnum((unsigned char)postfix[i]))
      {
         push(createNode(postfix[i]));
```

```
        }
else
        {
            right = pop();          left =
pop();          root =
createNode(postfix[i]);
root->left = left;          root->right =
right;          push(root);
        }
    }
    root = pop();     printf("Expression tree constructed
successfully.\n");
    return 0;
}
```

OUTPUT:

Enter postfix expression: ab-cd*+

Expression tree constructed successfully.

2. Write a C program to check whether a given binary tree is a Binary Search Tree (BST).

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <limits.h>
struct node
{    int data;    struct node
*left, *right;
};
struct node *createNode(int data)
{
   struct node *newnode = (struct node *)malloc(sizeof(struct node));
newnode->data = data;    newnode->left = NULL;    newnode->right =
NULL;    return newnode;
}
int isBST(struct node *root, long long min, long long max)
{
   if (root == NULL)        return 1;    if
(root->data <= min || root->data >= max)
       return 0;
   return isBST(root->left, min, root->data) &&
isBST(root->right, root->data, max);
}
```

```
int main()

{

   struct node *root;     root =

createNode(50);     root->left =

createNode(30);     root->right =

createNode(70);     root->left->left =

createNode(20);     root->left->right =

createNode(40);     root->right->left =

createNode(60);     root->right->right =

createNode(80);     if (isBST(root,

LLONG_MIN, LLONG_MAX))        printf("The

binary tree is a BST.\n");     else

printf("The binary tree is not a BST.\n");

return 0;

}
```

OUTPUT:

The binary tree is a BST.

3. Write a C program to implement a Max Heap.

```
#include <stdio.h> int heap[100]; int n = 0; void

insert(int value)
```

```
{    int i,
temp;    i =
n;
   heap[n] = value;    n++;    while (i > 0
&& heap[(i - 1) / 2] < heap[i])
   {
      temp = heap[i];
heap[i] = heap[(i - 1) / 2];
heap[(i - 1) / 2] = temp;
      i = (i - 1) / 2;
   }
}
void display()
{    int
i;
printf(
"Max
Heap:
");
for (i =
```

```
0; i <
n; i++)
printf(
"%d ",
heap[i
]);
printf(
"\n");
}
int main()
{
   int count, value, i;     printf("Enter
number of elements: ");     scanf("%d",
&count);     printf("Enter elements: ");
for (i = 0; i < count; i++)
   {
      scanf("%d", &value);
insert(value);
   }
```

```
   display();
return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter elements: 40 20 30 10 50

Max Heap: 50 40 30 10 20

4. Given an undirected graph the task to check, if there is a cycle in the given graph.

```
#include <stdio.h> int
graph[20][20], visited[20], n; int
hasCycle(int v, int parent)
{
   int i;    visited[v] =
1;    for (i = 0; i < n;
i++)
   {
      if (graph[v][i])
      {          if
(!visited[i])
```

```
        {           if
(hasCycle(i, v))
return 1;
        }
        else if (i != parent)
return 1;

      }
   }
   return 0;
}
int main()
{    int e, u, v,
i;
   printf("Enter number of vertices and edges: ");
scanf("%d%d", &n, &e);    printf("Enter edges (u
v):\n");    for (i = 0; i < e; i++)
   {
      scanf("%d%d", &u, &v);
graph[u][v]          =          1;
graph[v][u] = 1;
```

```
    }
    for (i = 0; i < n; i++)
visited[i] = 0;     for (i
= 0; i < n; i++)
    {
        if (!visited[i] && hasCycle(i, -1))
        {
            printf("Cycle exists in the graph.\n");
return 0;
        }
    }
    printf("No cycle exists in the graph.\n");
return 0;
}
```

OUTPUT:

Enter number of vertices and edges: 3 3

Enter edges (u v):

0 1

1 2

2 0

Cycle exists in the graph.

5. Given an grid consisting of 'L' (Land) and 'W' (Water), count the number of islands. #include <stdio.h> char grid[5][5] = {

```
   {'L','L','W','W','W'},
   {'L','L','W','W','L'},
   {'L','W','W','L','L'},
   {'W','W','W','W','W'},
   {'L','W','L','L','W'}
};
void dfs(int r, int c)
{
   if (r < 0 || r >= 5 || c < 0 || c >= 5 ||
grid[r][c] == 'W')        return;
grid[r][c] = 'W';     dfs(r + 1, c);     dfs(r -
1, c);     dfs(r, c + 1);     dfs(r, c - 1);
}
int main()
{     int i, j, count = 0;
for (i = 0; i < 5; i++)
```

```
    {        for (j = 0; j < 5;
j++)        {            if
(grid[i][j] == 'L')
            {
                count++;
dfs(i, j);
            }
        }
    }
    printf("Number of islands = %d", count);
return 0;
}
```

OUTPUT:

Number of islands = 4

6. Find out the Minimum Spanning Tree using Kruskal's algorithm.

```
#include <stdio.h> struct Edge
{    int u, v,
w;
};
```

```
struct Edge e[100], temp; int
parent[20];

int find(int x)
{
   while (parent[x] != x)
x = parent[x];     return
x;
}
int main()
{    int n, m, i, j, u, v, total = 0, count = 0;
printf("Enter vertices and edges: ");
scanf("%d%d", &n, &m);    printf("Enter edges
(u v weight):\n");    for (i = 0; i < m; i++)
scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);
for (i = 0; i < n; i++)        parent[i] = i;    for (i =
0; i < m - 1; i++)        for (j = 0; j < m - i - 1; j++)
if (e[j].w > e[j + 1].w)
          {
             temp = e[j];
```

```
            e[j] = e[j + 1];
e[j + 1] = temp;
          }
   printf("Edges in MST:\n");     for (i =
0; i < m && count < n - 1; i++)
   {
      u = find(e[i].u);
v = find(e[i].v);      if
(u != v)
      {
         parent[u]         =         v;
printf("%d   -   %d   :   %d\n",
e[i].u, e[i].v, e[i].w);          total
+= e[i].w;          count++;
      }
   }
   if (count == n - 1)
printf("Minimum cost: %d\n", total);
   else        printf("Spanning tree does not
exist\n"); return 0;
```

}

OUTPUT:

Enter vertices and edges: 4 5

Enter edges (u v weight):

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Edges in MST:

2 - 3 : 4

0 - 3 : 5

0 - 1 : 10

Minimum cost: 19

7. Use Dijkstra's algorithm to find the shortest path from a source to all nodes.

```
#include <stdio.h>

#define INF 99999

int main()

{
```

```
   int g[20][20], dist[20], visited[20];
int n, source, i, j, u, min;
printf("Enter number of vertices: ");
scanf("%d", &n);     printf("Enter
adjacency matrix:\n");
   for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
       {
          scanf("%d", &g[i][j]);
if (i != j && g[i][j] == 0)
g[i][j] = INF;
       }
   printf("Enter source vertex: ");
scanf("%d", &source);
   for (i = 0; i < n; i++)
   {
       dist[i] = g[source][i];
visited[i] = 0;
   }
```

```
   dist[source]            =            0;
visited[source] = 1; for (i = 1; i <
n; i++)
   {
      min = INF;       u = -1;       for (j
= 0; j < n; j++)           if (!visited[j]
&& dist[j] < min)
         {               min
= dist[j];               u =
j;
         }
      if (u == -1)           break;
visited[u] = 1;       for (j = 0; j < n; j++)
if (!visited[j] && g[u][j] != INF &&
dist[u] + g[u][j] < dist[j])              dist[j]
= dist[u] + g[u][j];
   }
   printf("Shortest distances from source %d:\n", source);
   for (i = 0; i < n; i++)
   {
```

```
        if (dist[i] == INF)            printf("%d -> %d :
Unreachable\n", source, i);
        else            printf("%d -> %d : %d\n", source,
i, dist[i]);
    }
    return 0;
}
```

OUTPUT:

Enter number of vertices: 4

Enter adjacency matrix:

0 4 1 0

4 0 2 5

1 2 0 8

0 5 8 0

Enter source vertex: 0

Shortest distances from source 0:

0 -> 0 : 0

0 -> 1 : 3

0 -> 2 : 1

0 -> 3 : 8

8. Write a program to find an Mother Vertex in the graph.

A Mother Vertex in a graph G=(V,E) is a vertex V such that all other vertices in G can be reached by a pain from V.

```
#include <stdio.h> int
graph[20][20], visited[20], n;
void DFS(int v)
{ int i; visited[v]
= 1; for (i = 0; i < n;
i++)
  {
    if (graph[v][i] && !visited[i])
      DFS(i);
  }
}
int main()
{
  int i, j, candidate = -1, found;
printf("Enter number of vertices: ");
scanf("%d", &n); printf("Enter
adjacency matrix:\n");
  for (i = 0; i < n; i++)
```

```
      for (j = 0; j < n; j++)
scanf("%d", &graph[i][j]);    for (i =
0; i < n; i++)
   {       for (j = 0; j < n;
j++)           visited[j] =
0;       DFS(i);
found = 1;       for (j =
0; j < n; j++)
      {           if
(!visited[j])
         {
            found = 0;
break;
         }
      }
      if (found)
      {
         candidate = i;
break;
      }
```

```
    }
    if (candidate != -1)        printf("Mother
Vertex: %d\n", candidate);
    else        printf("No Mother Vertex
exists.\n");     return 0;
}
```

OUTPUT:

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

0 0 0 1

0 0 0 1

0 0 0 0

No Mother Vertex exists.

9. A directed graph is strongly connected if there is a path between all pairs of vertices. A strongly connected component of a directed graph is a maximal strongly connected subgraph. Given a graph print all SCC.

```
#include   <stdio.h>   int
g[20][20],  t[20][20];  int
visited[20], stack[20]; int
```

```
n, top = -1; void dfs1(int
v)
{    int i;    visited[v] = 1;
for (i = 0; i < n; i++)       if
(g[v][i] && !visited[i])
dfs1(i);
   stack[++top] = v;
}
void dfs2(int v)
{    int i;    visited[v] = 1;
printf("%d ", v);    for (i =
0; i < n; i++)       if (t[v][i]
&& !visited[i])
dfs2(i);
}
int main()
{
   int i, j, v, component = 0;
printf("Enter number of vertices: ");
```

```
scanf("%d", &n);     printf("Enter
adjacency matrix:\n");
   for (i = 0; i < n; i++)
for (j = 0; j < n; j++)
      {
         scanf("%d", &g[i][j]);
t[j][i] = g[i][j];
      }
   for (i = 0; i < n; i++)
visited[i] = 0;     for (i
= 0; i < n; i++)        if
(!visited[i])
dfs1(i);     for (i = 0; i
< n; i++)
visited[i] = 0;
   printf("Strongly Connected Components:\n");
while (top != -1)
   {
      v = stack[top--];
      if (!visited[v])
```

```
        {
            component++;
printf("SCC %d: ", component);
            dfs2(v);
printf("\n");
        }
    }
    return 0;
}
```

OUTPUT: Enter number of

vertices: 4

Enter adjacency matrix:

0 1 0 0

0 0 1 0

1 0 0 0

0 0 0 0

Strongly Connected Components:

SCC 1: 3

SCC 2: 0 1 2

10. A bipartite graph G=(V,E) is a graph such that V can be partitioned into two subsets V_1 and V_2, and no edge has been its vertices in the same subset. Write a program to determine whether a graph is a bipartite graph

```
#include <stdio.h> int
graph[20][20], color[20], n; int
isBipartite()
{
   int queue[20];    int
front, rear, i, u, v;    for
(i = 0; i < n; i++)
color[i] = -1;    for (i =
0; i < n; i++)
   {       if (color[i] != -
1)          continue;
front = rear = 0;
queue[rear++] = i;
color[i] = 0;       while
(front < rear)
      {
         u = queue[front++];
         for (v = 0; v < n; v++)
```

```
        {
            if (graph[u][v])
            {                   if
(color[v] == -1)
                {
                    color[v] = 1 - color[u];
queue[rear++] = v;
                }
                else if (color[v] == color[u])

{                       return
0;
                }
            }
        }
      }
   }
   return 1;
}
int main()
```

```
{
   int i, j;
   printf("Enter number of vertices: ");
scanf("%d",   &n);            printf("Enter
adjacency matrix:\n");
   for (i = 0; i < n; i++)       for (j = 0; j <
n; j++)           scanf("%d",
&graph[i][j]);    if (isBipartite())
printf("Graph is Bipartite.\n");    else
printf("Graph is not Bipartite.\n");
return 0;
}
```

OUTPUT: Enter number of

vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Graph is Bipartite.